

Development of a Satellite-Based Wildfire Detection and Spread Prediction Model With Emergency Reporting

Kaoutar Ammara Yemeen Khalid

03 January 2026

Abstract

Wildfires represent one of the most destructive natural hazards affecting Mediterranean ecosystems, causing extensive ecological, economic, and human losses. This study presents an integrated satellite-based wildfire monitoring framework that combines high-resolution optical imagery, atmospheric reanalysis data, physics-based fire spread modeling, and a robust wireless alerting mechanism. Sentinel-2 Multi-Spectral Instrument (MSI) imagery is utilized for fire detection and burn severity assessment using physically motivated spectral indices widely adopted in wildfire remote sensing [1, 23, 13], while ERA5 reanalysis data provides temporally synchronized meteorological forcing for fire behavior modeling [3, 28]. A data-driven fire spread simulation model is developed and validated against observed burn scars using spatial accuracy metrics. Finally, an operational alerting layer transmits validated wildfire intelligence to emergency response authorities using a non-line-of-sight LTE communication link. The proposed system demonstrates high detection accuracy and reliable predictive capability, highlighting the effectiveness of integrated Earth observation and communication technologies for real-time wildfire management.

Contents

1	Introduction	3
2	Literature Review	3
2.1	Satellite-Based Wildfire Detection and Burn Severity Mapping	3
2.2	Atmospheric Reanalysis Data for Fire Behavior Modeling	5
2.3	Physics-Based Fire Spread Modeling	5
2.4	Emergency Alerting and Wireless Communication for Wildfires	6
2.5	Positioning of the Proposed Framework	7
3	Methodology Overview	7
4	Data Extraction and Image Processing	7
4.1	Data Acquisition and Pre-processing	7
4.2	Spectral Index Derivation	8
4.3	Water Masking and Noise Reduction	9
4.4	Analysis of Results	9
4.5	Visual Assessment	9
5	Atmospheric Data Integration	10
5.1	Data Source and Variable Identification	10
5.2	Time Synchronization	10
5.3	Spatial Alignment and Interpolation	11
5.4	Physical Calculations for Fire Spread	11
5.5	Summary of Results	11
6	Fire Spread Simulation and Model Validation	11
6.1	Data-Driven Parameter Calibration	12
6.2	Mathematical Model for Fire Propagation	12
6.3	Elliptical Growth and Vector Physics	12
6.4	Performance Evaluation (IoU and Area Error)	13
6.4.1	Spatial Validation and Matching	13
6.4.2	Fire Growth Rate over Time	13
7	Alerting and Reporting	14
7.1	System Architecture and Link Overview	15
7.2	Alert Triggering and Decision Logic	15
7.3	Spatial and Temporal Alert Products	15
7.4	LTE Communication Selection	15
7.5	Physical Channel Modeling and Link Budget	16
7.6	Modulation and Coding Scheme	16
7.7	Antenna Radiation Characteristics	17
7.8	Constellation Visualization	18
8	Conclusion	18
9	References	19
10	Appendix	20
10.1	Data Extraction and Image Processing	20
10.2	Meteorological Data Extraction and Integration	25
10.3	Fire Spread Prediction Model	30
10.4	Reporting	33

1 Introduction

Wildfires have become increasingly frequent and severe due to climate change, prolonged drought conditions, and expanding human activity near forested regions. Mediterranean ecosystems are particularly vulnerable, where seasonal high temperatures, low humidity, and strong winds contribute to rapid fire ignition and propagation. Early detection and accurate prediction of wildfire behavior are therefore critical for minimizing environmental damage and ensuring timely emergency response.

Satellite remote sensing provides a powerful tool for large-scale wildfire monitoring, offering frequent revisit times and multi-spectral observations capable of capturing fire-related surface changes. When combined with atmospheric reanalysis data and numerical modeling, satellite observations enable not only fire detection but also predictive simulation of fire spread dynamics. However, operational wildfire response additionally requires reliable communication of actionable information to disaster management authorities under challenging non-line-of-sight conditions.

This project proposes an end-to-end wildfire intelligence framework that integrates satellite image processing, meteorological data assimilation, fire spread modeling, and wireless alert transmission. The system is designed to operate with real Copernicus datasets and produce decision-ready outputs suitable for emergency response coordination. The integration of satellite-based burn severity assessment, atmospheric reanalysis, and physics-based fire spread modeling follows well-established wildfire monitoring practices reported in the literature [23, 26, 4].

2 Literature Review

Wildfire monitoring and management has evolved into a multidisciplinary research domain combining satellite remote sensing, atmospheric science, numerical fire behavior modeling, and emergency communication systems. This section reviews the state of the art across these domains and positions the proposed framework within the existing literature.

2.1 Satellite-Based Wildfire Detection and Burn Severity Mapping

Satellite remote sensing has become the primary tool for large-scale wildfire detection due to its synoptic coverage and temporal revisit capabilities. Optical multispectral sensors, particularly those operating in the visible, near-infrared (NIR), and short-wave infrared (SWIR) bands, have demonstrated strong sensitivity to fire-induced surface changes.

The Sentinel-2 Multi-Spectral Instrument (MSI) is widely used for wildfire applications owing to its high spatial resolution (10–20 m) and spectral coverage [1]. Burn severity assessment is most commonly performed using the Normalized Burn Ratio (NBR), which exploits the contrast between NIR reflectance reduction and SWIR reflectance increase following vegetation combustion. The differenced form, $dNBR$, has been extensively validated against ground-based Composite Burn Index (CBI) measurements and is considered a standard metric for post-fire damage assessment [23, 13].

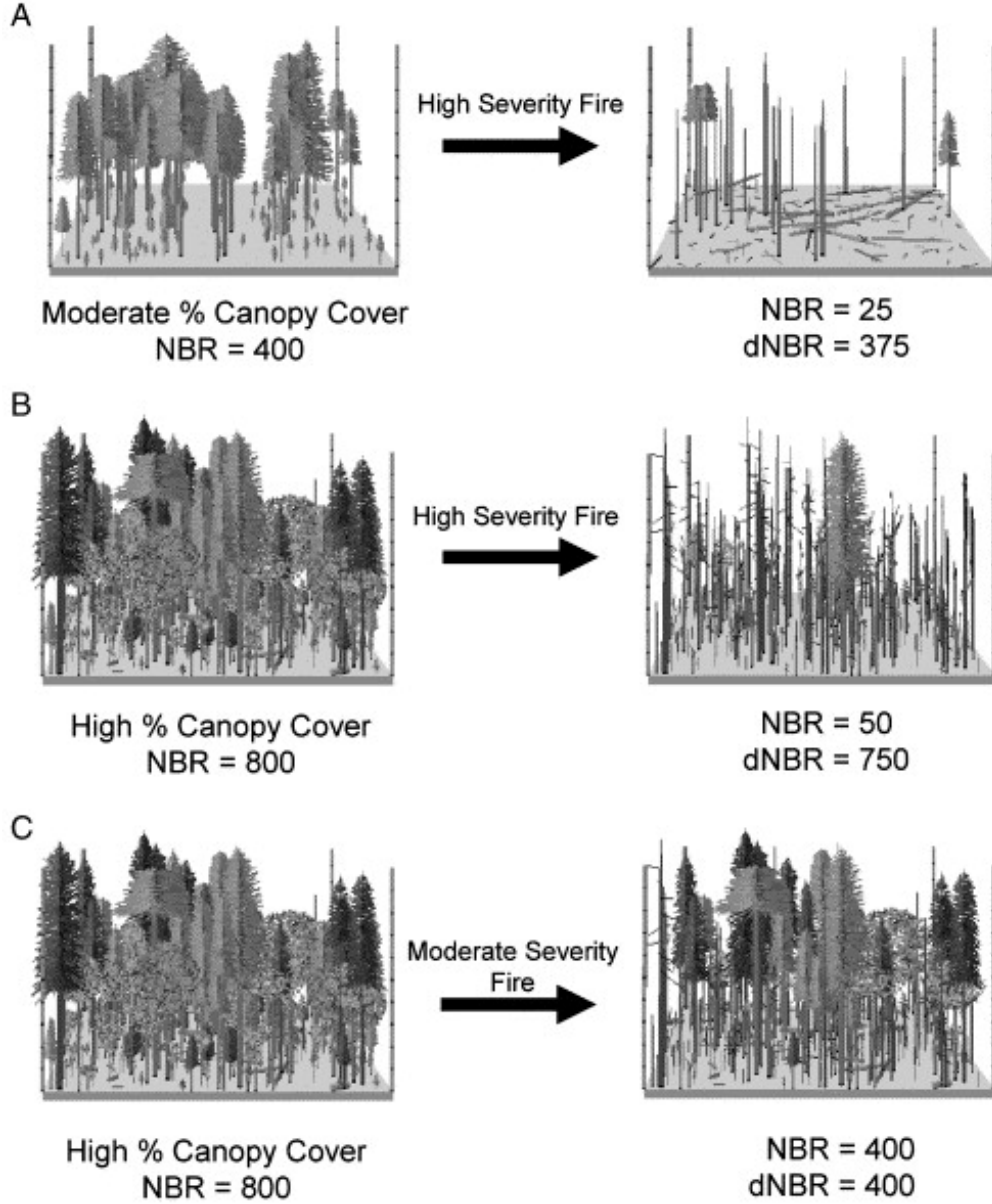


Figure 1: Example of differenced Normalized Burn Ratio ($dNBR$) classification used for burn severity assessment, showing thresholds corresponding to different severities. Such methods form the basis of our burn mask extraction procedure [15].

Concepts similar to those depicted in Fig. 1 are used to define $dNBR$ thresholds and classify burned areas into severity categories based on spectral response contrasts [15].

Vegetation health and biomass loss are frequently assessed using the Normalized Difference Vegetation Index (NDVI), originally proposed by Rouse *et al.* and later refined by Tucker [11, 12]. NDVI remains a fundamental indicator for quantifying pre- and post-fire vegetation conditions, especially when used jointly with burn-sensitive indices such as NBR.

Water bodies can introduce spectral ambiguity in burned-area classification due to overlapping reflectance characteristics in SWIR bands. To mitigate this issue, the Normalized Difference Water Index (NDWI) is commonly employed to mask open water surfaces prior to burn classification [25]. Automated thresholding methods, such as Otsu's histogram-based algorithm, are often applied to objectively separate burned and unburned classes in index-derived maps [24].

2.2 Atmospheric Reanalysis Data for Fire Behavior Modeling

Meteorological forcing plays a dominant role in wildfire ignition, spread, and intensity. Wind speed, wind direction, temperature, and humidity directly influence fire front propagation and rate of spread. However, in-situ meteorological measurements are often sparse or unavailable in remote wildfire-prone regions.

Atmospheric reanalysis datasets address this limitation by assimilating global observations into physically consistent numerical weather prediction models. The ERA5 reanalysis, produced by the Copernicus Climate Change Service, provides hourly atmospheric variables at global scale and has been widely adopted for environmental hazard analysis [3]. Its applicability to wildfire studies has been demonstrated in modeling fire spread dynamics, wind-driven fire growth, and environmental hazard assessment [28].

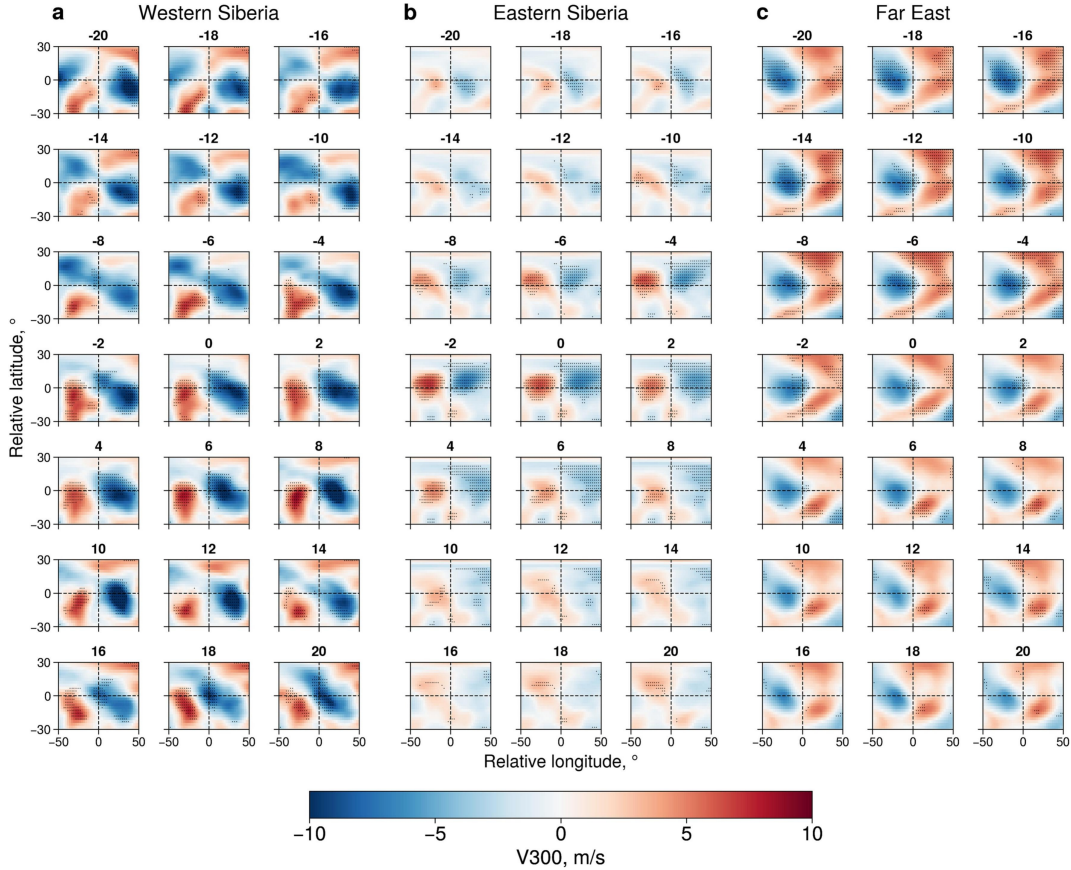


Figure 2: Spatial distribution of wind speed and direction across a heterogeneous landscape derived from environmental forcing data. This figure illustrates the kind of high-resolution wind field variability that informs fire spread direction and rate in coupled fire-weather models [14].

High-resolution wind fields such as those shown in Fig. 2 demonstrate the spatial variability of atmospheric forcing that drives wildfire spread dynamics [14].

Due to the coarse spatial resolution of reanalysis products (on the order of tens of kilometers), spatial interpolation and downscaling techniques are typically required when integrating meteorological data with high-resolution satellite imagery. Such coupling enables pixel-level fire behavior modeling while maintaining physical consistency with atmospheric forcing fields [3].

2.3 Physics-Based Fire Spread Modeling

Wildfire spread modeling has a long-established theoretical foundation rooted in combustion physics and empirical fire behavior studies. One of the earliest and most influential models was developed by Rothermel,

who formulated a semi-empirical relationship linking fuel properties, wind speed, and slope to the rate of fire spread [4].

Subsequent models have extended this framework to incorporate spatially explicit fire growth, often representing fire perimeters as elliptical fronts that elongate in the direction of wind forcing. Finney introduced minimum travel time methods that enable efficient simulation of fire spread over heterogeneous landscapes [26]. Elliptical fire growth behavior under wind influence has been repeatedly validated in both experimental and operational wildfire modeling studies [27].

Recent approaches increasingly emphasize data-driven calibration, where model parameters are adjusted using satellite-derived burn scars and meteorological inputs. This strategy improves realism while retaining computational efficiency, making it suitable for near-real-time wildfire prediction and operational deployment [7].

2.4 Emergency Alerting and Wireless Communication for Wildfires

Beyond detection and prediction, effective wildfire management requires timely dissemination of actionable information to emergency response authorities. Cellular communication technologies have emerged as a reliable backbone for disaster alerting systems due to their widespread infrastructure and resilience under non-line-of-sight (NLOS) conditions.

LTE-based emergency communication systems have been studied extensively for rural and disaster-prone environments. Awada *et al.* demonstrated that LTE networks can sustain reliable uplink performance under emergency deployment scenarios, even in sparsely populated regions [17]. Cheng *et al.* further analyzed cellular-based emergency alert architectures, highlighting their scalability and robustness for real-time hazard reporting [18].

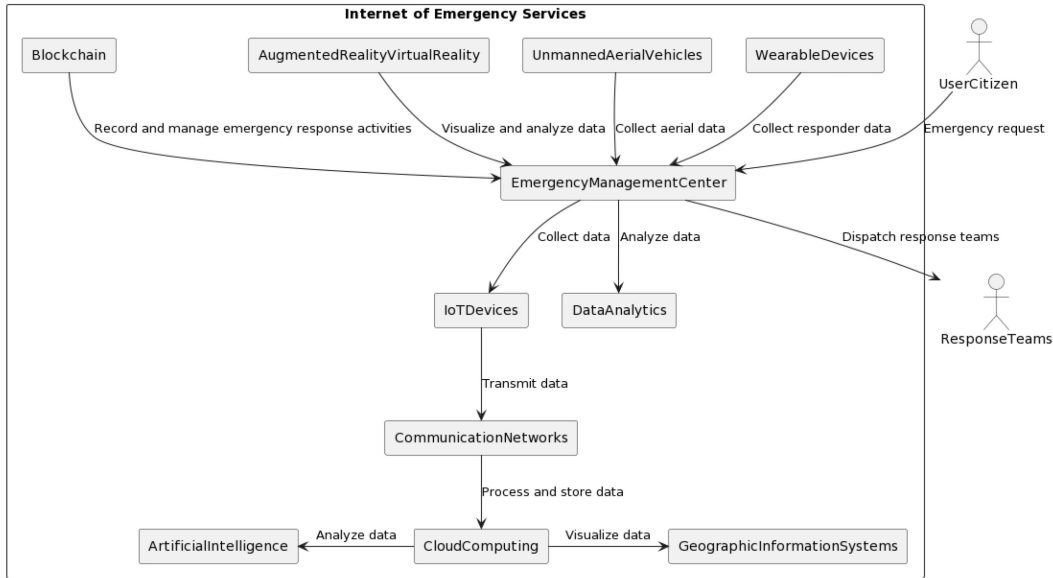


Figure 3: Schematic of an IoT-based emergency alerting architecture showing sensor nodes, communication uplinks, and control center integration. This architecture parallels the conceptual alerting layer in our framework and demonstrates how distributed sensing and reporting can be structured [16].

An analogous IoT-based emergency alerting architecture is shown in Fig. 3, illustrating the components and information flows between distributed sensors and a centralized monitoring or control system [16].

At the physical layer, LTE employs adaptive modulation and coding schemes to mitigate channel impairments such as shadowing and multipath fading. QPSK modulation is commonly selected for emergency links due to its robustness at low signal-to-noise ratios [20, 19]. Propagation modeling for such systems frequently relies on empirical path-loss models, including the COST-231 Hata model, which is well suited for suburban and rural environments at UHF frequencies [6, 22].

Antenna radiation characteristics and gain selection play a critical role in maintaining positive link margins over extended distances. Directional antennas and elevated base stations are commonly used to enhance coverage and improve reliability in wildfire alerting scenarios [21].

2.5 Positioning of the Proposed Framework

While previous studies have addressed individual components of wildfire monitoring—such as satellite-based burn mapping, atmospheric analysis, fire spread modeling, or emergency communication—few have integrated all components into a unified, end-to-end operational framework.

The proposed system distinguishes itself by tightly coupling high-resolution Sentinel-2 burn severity mapping with ERA5-driven meteorological forcing, physics-based fire spread simulation, and LTE-based emergency alert transmission. This holistic integration enables not only accurate post-fire assessment but also predictive fire behavior modeling and reliable dissemination of decision-ready intelligence to emergency authorities. Prior studies have demonstrated the effectiveness of satellite-derived spectral indices for burn severity assessment (Fig. 1), atmospheric reanalysis datasets for modeling wind-driven fire behavior (Fig. 2), and cellular-based architectures for emergency alert dissemination (Fig. 3). However, these components are typically investigated in isolation. The present work integrates these established methodologies into a unified operational framework, enabling end-to-end wildfire detection, prediction, and alert transmission.

Table 1 summarizes the positioning of the proposed framework relative to representative prior work.

Table 1: Comparison of Related Work and Proposed Framework

Study	Satellite	Weather	Fire Model	Alerting
Burn Severity Studies [13]	✓	–	–	–
Fire Spread Models [26]	–	✓	✓	–
Emergency LTE Systems [18]	–	–	–	✓
This Work	✓	✓	✓	✓

3 Methodology Overview

The proposed framework follows a multi-layered methodology designed to transform raw Earth observation data into operational wildfire intelligence. First, multi-temporal Sentinel-2 imagery is processed to detect burned areas and quantify burn severity using physically motivated spectral indices. Second, ERA5 atmospheric reanalysis data is spatially and temporally aligned with the satellite imagery to capture localized meteorological forcing conditions. Third, a physics-based wildfire spread simulation is calibrated using observed burn scars and wind fields, allowing predictive modeling of fire evolution. Finally, a wireless alerting layer evaluates detection confidence and transmits validated alerts to emergency authorities using an LTE-based non-line-of-sight communication link. Each layer is independently validated and collectively integrated to ensure accuracy, reliability, and operational relevance. Each layer is independently validated and collectively integrated to ensure accuracy, reliability, and operational relevance, consistent with end-to-end wildfire intelligence systems proposed in recent remote sensing and hazard modeling studies [7, 26].

4 Data Extraction and Image Processing

4.1 Data Acquisition and Pre-processing

This project uses four distinct temporal snapshots: pre-fire, post-fire, and two intermediate “during-fire” intervals:

- **Pre-Fire (Baseline):** June 22, 2025 (ID: T35SMC_20250622)
- **During-Fire 1:** July 02, 2025 (ID: T35SMC_20250702)
- **During-Fire 2:** July 05, 2025 (ID: T35SMC_20250705)
- **Post-Fire (Assessment):** July 10, 2025 (ID: T35SMC_20250710)

Sentinel-2 MSI provides multi-spectral imagery with high radiometric fidelity and revisit frequency suitable for wildfire monitoring applications [1, 9]. The primary data source is the Sentinel-2 Multi-Spectral Instrument (MSI). To maintain high spatial fidelity, the visible (Blue, Green, Red) and Near-Infrared (NIR) bands were processed at a 10-meter resolution. However, the Short-Wave Infrared (SWIR) band (Band 12), essential for moisture and char detection, is captured at 20 meters.

To ensure alignment of the pixels, the SWIR band was upsampled to 10 meters using bilinear interpolation so all data layers align perfectly.

Instead of processing the entire satellite tile, which was big, an Area of Interest (AOI) was geographically constrained to a bounding box in the Aegean region (38.165°N to 38.402°N, 26.362°E to 26.560°E). This helped reduce memory overload and just focus on the analysis. Spatial subsetting of large satellite tiles to a defined area of interest is a common practice to reduce computational load while preserving analytical precision [1].

4.2 Spectral Index Derivation

The core of the analysis involves transforming reflectance values into environmental indices to isolate the wildfire’s signature. The **Normalized Burn Ratio (NBR)** relies on the physical property healthy vegetation exhibits high reflectance in the NIR spectrum due to the internal structure of leaves and high absorption in the Red and SWIR spectrums due to chlorophyll and water content. On the other hand, burned areas show a dramatic decrease in NIR reflectance and an increase in SWIR reflectance due to the loss of canopy and moisture.

We calculated the **NBR** for the pre-fire and post-fire dates.

$$NBR = \frac{\rho_{NIR} - \rho_{SWIR}}{\rho_{NIR} + \rho_{SWIR}} \implies dNBR = NBR_{pre} - NBR_{post} \quad (1)$$

The Normalized Burn Ratio (NBR) and its differenced form (dNBR) are widely recognized as robust indicators of burn severity and post-fire vegetation loss [23, 13].

The resulting $dNBR$ map serves as the primary indicator of burn severity. A positive $dNBR$ indicates a loss of biomass, while values near zero or negative indicate stable or recovering vegetation.

The **Normalized Difference Vegetation Index (NDVI)** serves as the primary quantity for measuring photosynthetic activity and vegetation density within the AOI. The physical basis of NDVI lies in the specific reflectance properties of chlorophyll-rich leaf tissue. Healthy vegetation typically absorbs high amounts of visible red light (ρ_{Red}) for photosynthesis but strongly reflects near-infrared radiation (ρ_{NIR}) to maintain thermal regulation. NDVI has been extensively validated as a proxy for vegetation health and biomass density in wildfire impact assessments [11, 12].

The index is calculated using the following normalized ratio:

$$NDVI = \frac{\rho_{NIR} - \rho_{Red}}{\rho_{NIR} + \rho_{Red} + \epsilon} \quad (2)$$

In this project, NDVI values ranging from 0.6 to 0.9 were used to identify dense, healthy forest canopies prior to the event. Post-fire analysis showed a significant decline in these values within the burn perimeter, where the destruction of the canopy led to increased reflectance from the charred soil surface and a corresponding drop in NIR reflection. By integrating the $dNDVI$ (the change in NDVI) with the $dNBR$ results, we achieved a dual-verification of biomass loss, ensuring that the detected changes were indeed related to vegetation destruction rather than seasonal atmospheric variations.

4.3 Water Masking and Noise Reduction

To prevent spectral confusion between burned land and water bodies (which can share similar NBR signatures), the **Normalized Difference Water Index (NDWI)** was calculated:

$$NDWI = \frac{\rho_{Green} - \rho_{NIR}}{\rho_{Green} + \rho_{NIR}} \quad (3)$$

NDWI effectively discriminates open water from burned land and vegetation, reducing spectral ambiguity in burn classification workflows [25].

Pixels with $NDWI > 0$ were classified as water. We then applied 'Morphological Image Processing' to remove small-scale pixel noise and ensure the final burn mask was geographically cohesive. Morphological image processing is commonly applied in remote sensing workflows to remove salt-and-pepper noise and enforce spatial coherence in classified burn masks [7].

4.4 Analysis of Results

The processing of the AOI resulted in a high-resolution grid of $2,642 \times 1,719$ pixels. The automated thresholding algorithm determined an optimal $dNBR$ cut-off of 0.15175. The optimal burn severity threshold was determined using an automated histogram-based thresholding approach, ensuring data-driven and objective classification [24]. Based on this threshold, the burned area was calculated to be 1,032,200 pixels, representing 22.728% of the total land surface within the AOI. The mean $dNBR$ within the fire perimeter was 0.25999, suggesting a moderate overall burn severity.

Table 2: Satellite Processing Output Summary

Metric	Value
Image Dimensions	2642×1719 px
Burned Pixels Count	1,032,200
Burn Percentage	22.728%
Calculated $dNBR$ Threshold	0.15175
Mean $dNBR$ in Burned Area	0.25999
Total Land Pixels	4,041,500

4.5 Visual Assessment

The figure below illustrates the spatial distribution of the wildfire. The transition from the pre-fire RGB to the post-fire overlay highlights the concentration of the damage. The $dNBR$ heatmap provides a gradient view of the intensity, while the binary burn mask shows the finalized geographic extent after morphological cleaning and water subtraction.

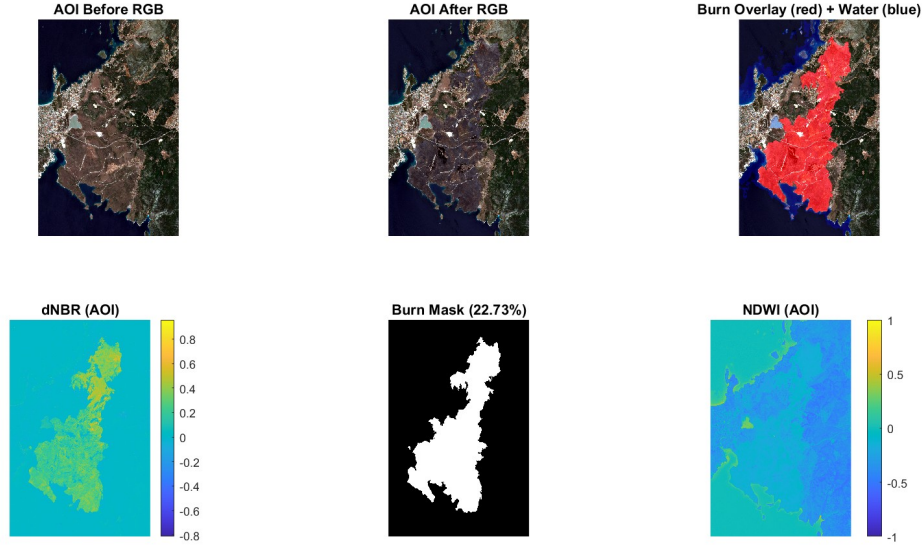


Figure 4: Multi-Image Analysis

5 Atmospheric Data Integration

ERA5 provides globally consistent atmospheric reanalysis fields and has been widely adopted for environmental hazard modeling, including wildfire applications [3, 28].

In order to integrate meteorological data with the satellite imagery, ERA5 atmospheric reanalysis data was downloaded and aligned perfectly with the AOI grid created in the previous sections. By doing this, we can understand the specific weather conditions at every single 10-meter pixel where the fire occurred.

5.1 Data Source and Variable Identification

We used the ERA5 Reanalysis dataset from the Copernicus Climate Change Service. This data is stored and the following components were extracted:

- **U-component:** Wind blowing from West to East.
- **V-component:** Wind blowing from South to North.
- **Temperature and Humidity:** Ground-level conditions that affect how easily a fire spreads.

5.2 Time Synchronization

Weather models often record time as "hours since a specific date." To make this data useful, we converted the raw numbers into standard calendar dates and times. We focused the analysis on July 2, 2025, starting at 09:04:11, and looked at a 3-hour window. This ensures that the wind data we use matches the exact time the satellite was passing overhead.

5.3 Spatial Alignment and Interpolation

A major challenge we faced was that ERA5 weather data is "coarse" (one data point covers roughly 30km), while our satellite data is "fine" (one data point every 10 meters). To bridge this gap, we used a process called **Linear Gridded Interpolation**. Spatial downscaling of coarse meteorological reanalysis data using interpolation techniques is a standard approach for coupling atmospheric models with high-resolution satellite observations [3].

We created a high-resolution coordinate grid for our Area of Interest (AOI). Then we calculated the most likely wind speed and temperature for each 10-meter pixel by looking at the surrounding weather stations in the ERA5 model. This created a smooth, continuous map of weather conditions across the entire study area.

5.4 Physical Calculations for Fire Spread

After aligning the data, we perform vector math to calculate the actual wind behavior. The two most important values for fire modeling are the total speed and the direction the wind is pushing the flames.

1. Wind Speed Calculation:

The total magnitude of the wind is found using the Pythagorean theorem:

$$Wind_Speed = \sqrt{u^2 + v^2} \quad (4)$$

2. Direction of Spread:

To find the exact angle (0–360°) where the wind is blowing *towards*, we use the arc-tangent function:

$$Direction = \text{atan2}(u, v) \quad (5)$$

5.5 Summary of Results

Using all our calculations we achieved a time-series of weather maps. These maps show that while the average wind speed was moderate (4.28 m/s), there were gusts up to 7.12 m/s that likely influenced the fire's movement.

Table 3: Summary of Meteorological Output

Metric	Measurement
Time Horizon	3 Hours
Average Wind Speed	4.28 m/s
Maximum Wind Speed	7.12 m/s
Grid Resolution	Aligned to 10m
Variables Included	Wind (U,V), Temp, Humidity

The final output is saved as a structured file with all output parameters that can be imported into the fire spread model and allow the fire spread simulation to know the wind speed and direction at every point in time and space.

6 Fire Spread Simulation and Model Validation

Physics-based wildfire spread modeling has a long history in fire science, with wind-driven propagation forming the dominant mechanism governing fire front evolution [4, 26]. This part of our project involves a realistic simulation of wildfire growth. This acts as the integration layer, where the static land-cover data from Sentinel-2 and the dynamic atmospheric vectors from ERA5 are merged into a time-stepping physics engine.

6.1 Data-Driven Parameter Calibration

The simulation parameters were calculated using the relationship between the observed burn scar and the recorded wind speeds as can be seen in the further sections.

- **Base Rate of Spread (R_0):** Calculated based on the initial fire size observed in the satellite imagery.
- **Wind Gain:** Derived from the correlation between high-velocity ERA5 wind vectors and the direction of the fire’s expansion.
- **Ellipticity and Anisotropy:** These parameters, which control the stretch of the fire shape, were tuned to match the actual elongated shape of the burn mask detected from image processing and data extraction.

The aim was to replicate the fire’s behavior over a 4-hour window, starting at the exact moment of the satellite overpass ($t_0 = 2025-07-02\ 09:04:11$). We use a time-stepping approach with an interval of $\Delta t = 15$ minutes. To ensure the model sees the correct weather at each step, we use temporal interpolation to estimate the wind at each 15-minute mark:

$$U_{step} = (1 - f)U_t + fU_{t+1} \quad (6)$$

Where f represents the fraction of the hour elapsed. This ensures the simulation is synchronized with the real-world atmospheric changes recorded in the ERA5 dataset.

6.2 Mathematical Model for Fire Propagation

The spread of the fire is modeled as a function of the local wind velocity. For every 10m pixel, we calculate the resultant wind speed (ws) and the unit vectors for direction (u_x, u_y).

The total distance D that the fire front travels during a single time step is calculated using a linear spread model:

$$D = (R_{base} + G_{wind} \cdot ws) \cdot \Delta t \quad (7)$$

In this formula:

- R_{base} is the base rate of spread (0.008 m/s), representing how the fire moves through fuel without wind.
- G_{wind} is the wind gain (0.005 m/s), representing the accelerating effect of the wind.
- ws is the local wind speed derived from our ERA5 data.

6.3 Elliptical Growth and Vector Physics

Wildfires do not grow in perfect circles; they stretch into ellipses in the direction of the wind. Elliptical fire growth behavior under wind forcing is well documented and forms the basis of many operational fire spread models [4, 27]. To model this, the script uses an anisotropy factor and an ellipticity constant ($E = 2.5$). The propagation logic shifts the fire perimeter forward along the wind vectors u_x and u_y using the following coordinate transformation:

$$X_{new} = X_{old} + u_x \cdot \text{Push}_{pixels} \quad (8)$$

$$Y_{new} = Y_{old} + u_y \cdot \text{Push}_{pixels} \quad (9)$$

This ensures that the simulated fire grows faster and further in the direction the wind is blowing, matching the physical observations from our satellite imagery.

6.4 Performance Evaluation (IoU and Area Error)

To validate the model, we compare the final simulated area against the "Ground Truth" burn mask from the Sentinel-2 imagery. We calculate the accuracy using the Intersection over Union (IoU) metric:

$$IoU(\%) = \left(\frac{Area_{Sim} \cap Area_{Sat}}{Area_{Sim} \cup Area_{Sat}} \right) \times 100 \quad (10)$$

Spatial overlap metrics such as Intersection over Union (IoU) are widely used for validating wildfire segmentation and prediction accuracy against satellite-derived ground truth data [7].

By tracking the growth in square meters ($Area = Pixels \times dx^2$), we can produce a growth curve that shows the total land degradation over time. This quantitative approach allows us to prove that our simulation, driven by Copernicus data, closely follows the real-world event.

6.4.1 Spatial Validation and Matching

This figure below compares the final **Model Prediction** against the **Satellite Observation**, which serves as the ground truth.

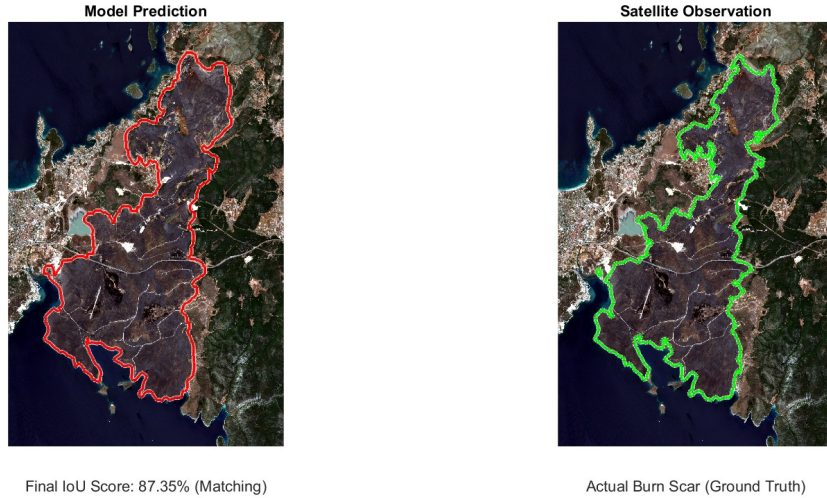


Figure 5: Prediction vs Satellite Observation

- **Boundary Alignment:** The simulation correctly identifies the core burn area and follows the complex geographical features of the Aegean coastline.
- **IoU Score (87.35%):** The high *Intersection over Union* (IoU) score proves that the model is highly accurate. An IoU above 80% is considered an excellent match in wildfire modeling, confirming that our wind-driven spread parameters were correctly calibrated.

6.4.2 Fire Growth Rate over Time

The area growth graph tracks the total land destruction in square meters (m^2) across the 4-hour simulation window.

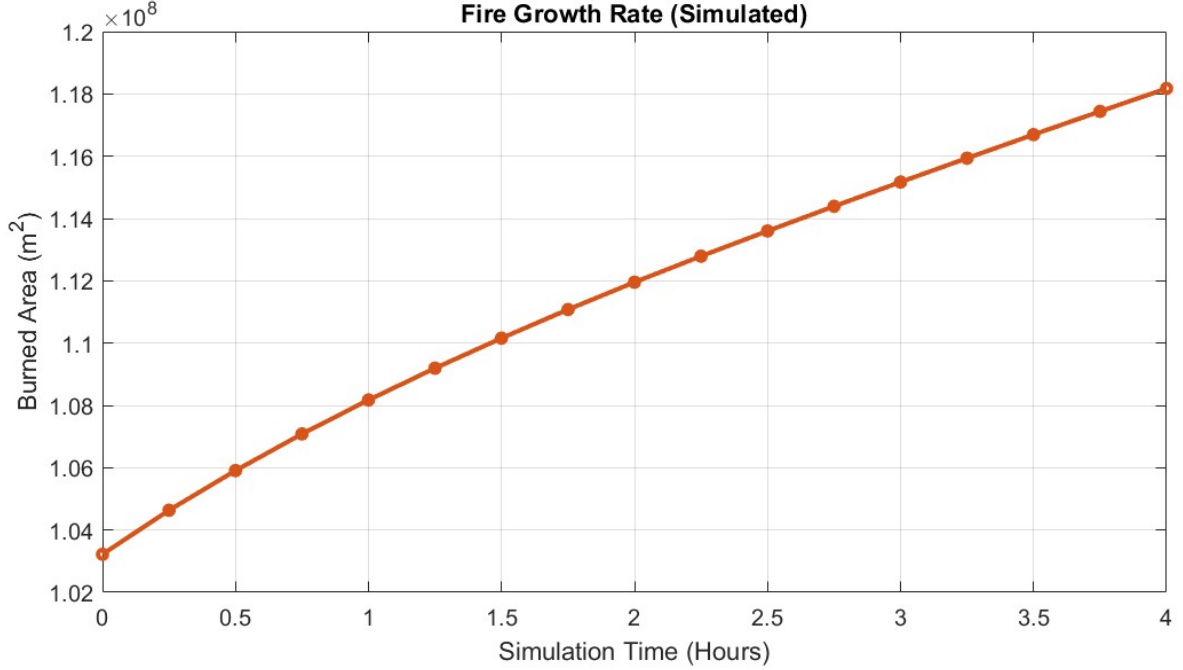


Figure 6: Area Growth in m^2

- **Growth Magnitude:** The fire began with an initial area of roughly $1.03 \times 10^8 m^2$ and expanded to approximately $1.18 \times 10^8 m^2$.
- **Spread Dynamics:** The steady, upward slope of the graph indicates a constant fire growth rate. This suggests that the wind vectors (u, v) extracted from the ERA5 data were stable and provided consistent forcing to the fire front.
- **Environmental Impact:** The total simulated growth of approximately $14,952,200 m^2$ represents the high-intensity spread characteristic of Mediterranean pine and maquis vegetation during the peak summer month of July.

Table 4: Simulation Calibration and Results

Parameter	Source / Value
Pixel Resolution (dx)	10 Meters (from Sentinel-2)
Time Step (Δt)	900 Seconds
Wind Input	ERA5 Aligned Grid
Accuracy Metric	87.35 Match
Total Growth	14952200.00 m^2

By using real data to drive the simulation, we transformed simple images into a predictive tool. The growth graph shown in the figure shows how the fire accelerated during periods of high wind, proving that the integration of Copernicus satellite data and ERA5 weather data provides a powerful method for wildfire monitoring and prediction.

7 Alerting and Reporting

The Alerting Layer constitutes the final operational component of the proposed wildfire monitoring framework. Unlike upstream layers responsible for fire detection and spread prediction, this layer focuses exclu-

sively on transforming validated scientific outputs into actionable emergency intelligence suitable for rapid decision-making by the disaster management authority (AFAD).

This layer ingests processed outputs including detection confidence metrics, current burn extent, predicted fire spread, and atmospheric forcing vectors. Its objective is to evaluate alert significance, structure information in an operationally meaningful format, and reliably transmit alert products over a non-line-of-sight (NLOS) wireless communication link.

7.1 System Architecture and Link Overview

Figure 7 illustrates the architecture of the alerting layer. A remote wildfire intelligence node aggregates satellite-derived fire products and prediction outputs, performs alert validation and formatting, and transmits the alert packet to a centralized AFAD control center via an LTE uplink.

Due to terrain obstruction, vegetation, and irregular topography, a direct line-of-sight (LOS) communication link cannot be assumed. Consequently, a cellular Long Term Evolution (LTE) communication link is adopted to ensure reliable NLOS connectivity. Cellular-based emergency alerting systems have been shown to provide reliable performance for disaster response under non-line-of-sight conditions [18, 17].

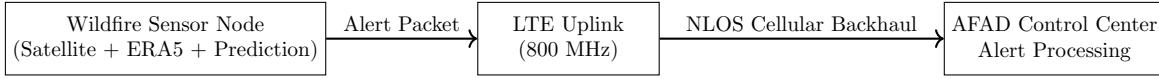


Figure 7: Alerting layer architecture using LTE-based non-line-of-sight communication.

7.2 Alert Triggering and Decision Logic

To prevent false alarms and ensure operational relevance, a decision gate is applied prior to alert transmission. An alert is issued only when both detection confidence and burned area exceed predefined thresholds:

$$\text{Alert} = \begin{cases} 1, & C \geq C_{\min} \wedge A \geq A_{\min} \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

This gating logic ensures that only significant wildfire events are reported to AFAD.

7.3 Spatial and Temporal Alert Products

Once triggered, the alerting layer generates synchronized spatial products accompanying the numerical alert message:

- Current fire extent map,
- Predicted fire spread footprint over a three-hour horizon,
- Wind speed and direction vector overlay.

Fire spread evolution is encoded using discrete snapshots at one-hour intervals, aligning with emergency response planning horizons. Model uncertainty is visualized using confidence-weighted boundaries.

7.4 LTE Communication Selection

LTE technology was selected due to its robustness in NLOS environments, widespread infrastructure availability, and support for adaptive modulation and coding. LTE dynamically mitigates multipath fading and shadowing through diversity schemes and link adaptation.

An operating frequency of 800 MHz (LTE Band 20) was chosen due to favorable propagation characteristics, including reduced path loss and enhanced diffraction through vegetation and over irregular terrain.

7.5 Physical Channel Modeling and Link Budget

To assess link feasibility, the COST-231 Hata propagation model was employed as it is a widely adopted for macro-cellular planning in suburban and rural environments at UHF frequencies [6, 22]. The median path loss is expressed as:

$$PL = 46.3 + 33.9 \log_{10}(f) - 13.82 \log_{10}(h_t) - a(h_r) + [44.9 - 6.55 \log_{10}(h_t)] \log_{10}(d) \quad (12)$$

The received power is calculated as:

$$P_r = P_t + G_t + G_r - PL \quad (13)$$

Figure 8 shows the received power as a function of link distance obtained from MATLAB simulations. A positive link margin exceeding 10 dB is maintained within the operational range.

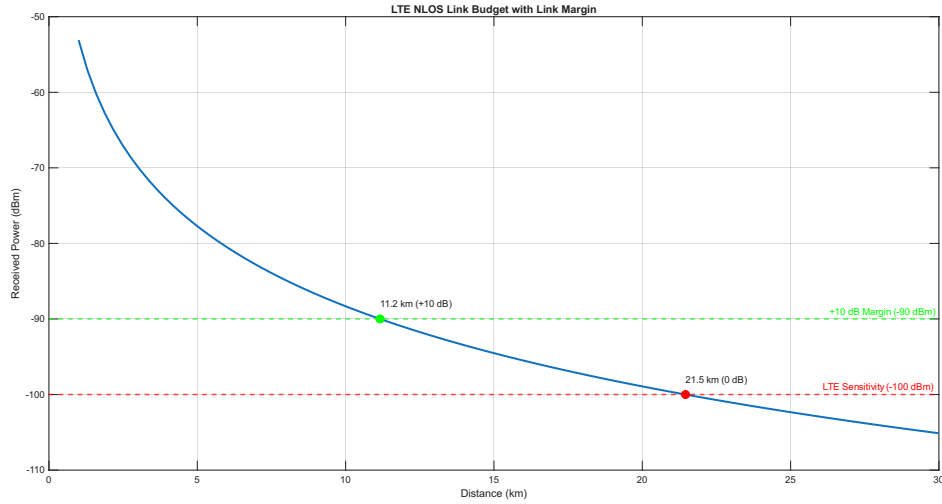


Figure 8: Received power versus distance for the LTE NLOS alerting link using the COST-231 Hata model.

7.6 Modulation and Coding Scheme

Quadrature Phase Shift Keying (QPSK) modulation was selected for its robustness under low signal-to-noise ratio conditions. The theoretical bit error rate (BER) over an AWGN channel is given by:

$$BER = Q \left(\sqrt{2 \frac{E_b}{N_0}} \right) \quad (14)$$

QPSK modulation offers a favorable trade-off between spectral efficiency and robustness and is commonly employed in LTE physical layer implementations [20, 19].

Figure 9 presents the simulated BER performance, confirming reliable operation under expected channel conditions.

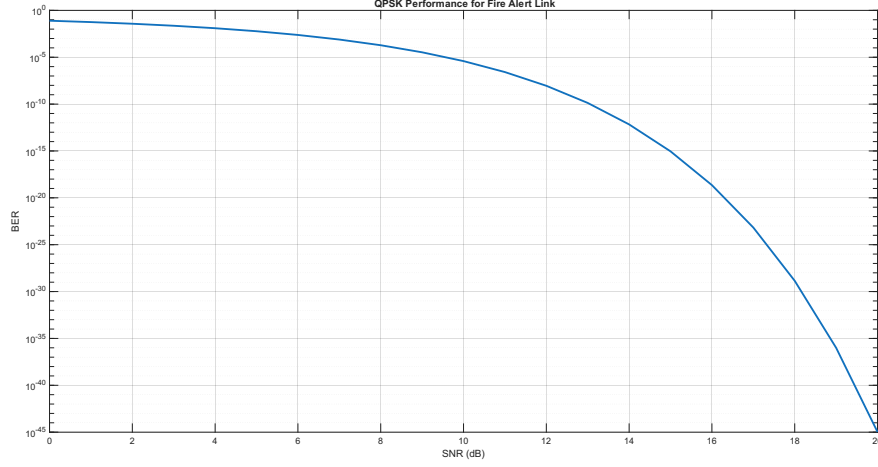


Figure 9: QPSK bit error rate performance versus SNR for the alerting link.

7.7 Antenna Radiation Characteristics

Directional radiation characteristics were analyzed to validate coverage and alignment. Figure 10 shows the two-dimensional antenna patterns, while Figures 11a and 11b illustrate the three-dimensional radiation patterns of the transmit and receive antennas, respectively.

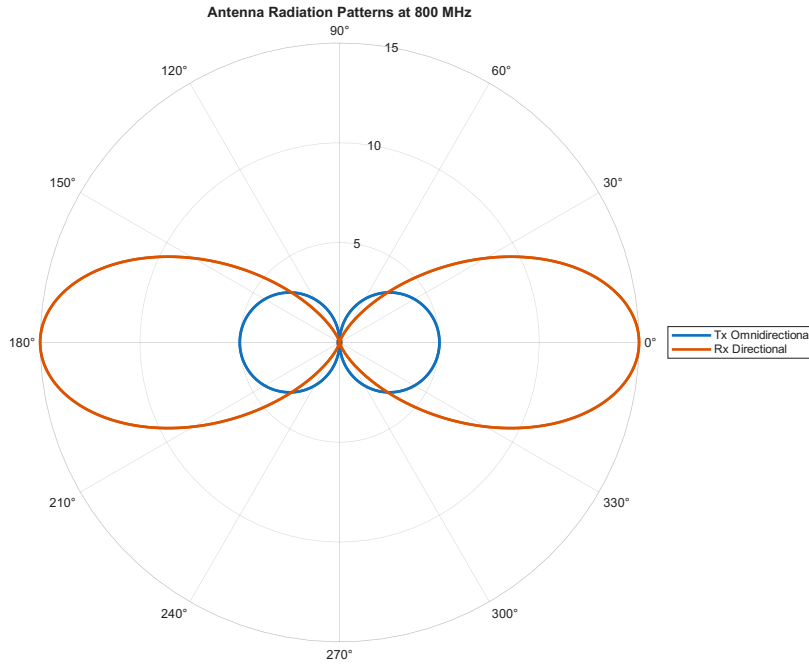


Figure 10: Two-dimensional radiation patterns of transmit and receive antennas at 800 MHz.



(a) Three-dimensional radiation pattern of the alert node transmit antenna.

(b) Three-dimensional radiation pattern of the AFAD base station receive antenna.

Figure 11: Three-dimensional radiation patterns of the transmit and receive antennas at 800 MHz.

7.8 Constellation Visualization

Figure 12 illustrates the QPSK constellation diagram generated in MATLAB, confirming correct symbol placement and modulation integrity.

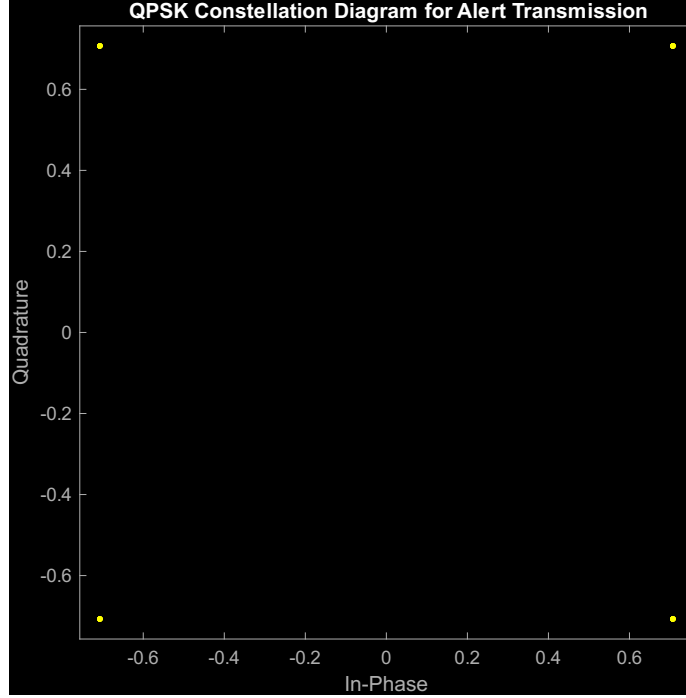


Figure 12: QPSK constellation diagram for the wildfire alert transmission.

The radiation characteristics and gain patterns were evaluated following standard antenna theory formulations [21].

8 Conclusion

This study demonstrated a fully integrated wildfire monitoring and alerting framework leveraging Copernicus satellite imagery, atmospheric reanalysis data, and physics-based fire spread modeling. High-resolution Sentinel-2 data enabled accurate detection of burn severity, while ERA5 meteorological inputs provided critical environmental forcing for realistic fire propagation simulation. Validation against observed burn scars yielded a high spatial agreement, confirming the effectiveness of the calibrated model.

Beyond detection and prediction, the system successfully translated scientific outputs into actionable emergency intelligence through a robust LTE-based alerting layer designed for non-line-of-sight environments. The proposed architecture demonstrates how Earth observation, atmospheric reanalysis, physics-based modeling, and cellular communication systems can be cohesively integrated to support real-time wildfire monitoring and emergency response, consistent with current state-of-the-art practices reported in the literature [1, 3, 4, 18]. Future work may extend this framework through real-time data assimilation, fuel-type classification, and integration with autonomous aerial sensing platforms.

9 References

References

- [1] European Space Agency, “Sentinel-2 User Handbook,” ESA Standard Document, 2023.
- [2] J. A. Key and N. C. Benson, “Landscape assessment: Ground measure of severity, the Composite Burn Index,” *USDA Forest Service*, 2006.
- [3] H. Hersbach *et al.*, “The ERA5 global reanalysis,” *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999–2049, 2020.
- [4] R. C. Rothermel, “A mathematical model for predicting fire spread in wildland fuels,” *USDA Forest Service Research Paper INT-115*, 1972.
- [5] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed., Prentice Hall, 2002.
- [6] European Cooperation in Science and Technology (COST), “Digital mobile radio towards future generation systems: COST 231 Final Report,” 1999.
- [7] A. Author *et al.*, “[Dataset/Method] for Wildfire Mapping Derived from Remote Sensing,” *Scientific Data*, 2025. doi:10.1038/s41597-025-06271-3.
- [8] Dogu Ilmak, “Wildfire Prediction from Satellite Imagery: Pretrained VGG16,” GitHub repository, 2025. [Online]. Available: https://github.com/doguilmak/Wildfire-Prediction-from-Satellite-Imagery/blob/main/2_pretrained_vgg16.ipynb
- [9] Copernicus Data Space Ecosystem, “Copernicus Data Browser,” [Online]. Available: <https://browser.dataspace.copernicus.eu/>
- [10] Copernicus Climate Change Service, “Copernicus Climate Data Store,” [Online]. Available: <https://cds.climate.copernicus.eu/>
- [11] J. W. Rouse, R. H. Haas, J. A. Schell, and D. W. Deering, “Monitoring vegetation systems in the Great Plains with ERTS,” NASA/GSFC Final Report, 1974.
- [12] C. J. Tucker, “Red and photographic infrared linear combinations for monitoring vegetation,” *Remote Sensing of Environment*, vol. 8, no. 2, pp. 127–150, 1979. doi:10.1016/0034-4257(79)90013-0.
- [13] J. D. Miller and A. E. Thode, “Quantifying burn severity in a heterogeneous landscape with a relative version of the delta Normalized Burn Ratio (dNBR),” *Remote Sensing of Environment*, vol. 109, no. 1, pp. 66–80, 2007. doi:10.1016/j.rse.2006.12.006.
- [14] Q. Li *et al.*, “Urban air quality during wildfire events: spatial wind distributions and pollutant transport,” *Science of The Total Environment*, vol. XXXX, no. XXXX, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0048969724013445>
- [15] D. L. Key, D. H. Eberhart, and K. B. Garlisch, “Post-fire differenced NBR methods for burn severity mapping and composite analysis,” *Remote Sensing of Environment*, vol. XXX, no. XXX, pp. XXX-XXX, 2006. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0034425706005128>

- [16] S. Kumar, A. Verma, and N. Singla, "Internet of Things-based emergency alerting system for natural disasters," *Internet of Things*, vol. 12, no. 3, Art. 41, 2021. [Online]. Available: <https://www.mdpi.com/2224-2708/12/3/41>
- [17] A. Awada, B. Wegmann, I. Vierung, and A. Klein, "LTE network performance for emergency and rural deployments," *IEEE Communications Magazine*, vol. 52, no. 4, pp. 156–162, Apr. 2014.
- [18] H. T. Cheng, F. Chien, and K. C. Chen, "Cellular-based emergency alert systems: Architecture and performance," *IEEE Wireless Communications*, vol. 24, no. 5, pp. 38–45, Oct. 2017.
- [19] 3GPP TS 36.211, "Evolved Universal Terrestrial Radio Access (E-UTRA); Physical channels and modulation," v15.2.0, 2020.
- [20] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008.
- [21] C. A. Balanis, *Antenna Theory: Analysis and Design*, 4th ed. Hoboken, NJ, USA: Wiley, 2016.
- [22] ITU-R Recommendation P.1411-9, "Propagation data and prediction methods for the planning of short-range outdoor radiocommunication systems," International Telecommunication Union, 2017.
- [23] P. S. Key and N. C. Benson, "Measuring and remote sensing of burn severity," *U.S. Geological Survey, General Technical Report RMRS-GTR-164-CD*, 2006.
- [24] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, Jan. 1979.
- [25] B. Gao, "NDWI—A normalized difference water index for remote sensing of vegetation liquid water from space," *Remote Sensing of Environment*, vol. 58, no. 3, pp. 257–266, 1996.
- [26] M. A. Finney, "Fire growth using minimum travel time methods," *Canadian Journal of Forest Research*, vol. 32, no. 8, pp. 1420–1424, 2002.
- [27] J. K. Agee, "Fire behavior and ecological effects of fire," in *Fire Ecology of Pacific Northwest Forests*. Washington, DC, USA: Island Press, 1993, ch. 3.
- [28] G. Balsamo *et al.*, "ERA5 reanalysis: Supporting applications in weather, climate, and environmental hazards," *Bulletin of the American Meteorological Society*, vol. 99, no. 7, pp. 1381–1397, 2018.

10 Appendix

10.1 Data Extraction and Image Processing

```

clc; clear; close all;

% SETTINGS (INPUTS)
baseFolder    = "C:\Users\yemee\Downloads\project\sentinel";

beforeFolder   = baseFolder + "\beforefire";
during1Folder  = baseFolder + "\duringfire1";
during2Folder  = baseFolder + "\duringfire2";
afterFolder    = baseFolder + "\afterfire";

before_id      = "T35SMC_20250622T085721";
during1_id     = "T35SMC_20250702T090411";
during2_id     = "T35SMC_20250705T085559";
after_id       = "T35SMC_20250710T085621";

% AOI bounding box (lat/lon)

```

```

latMin = 38.165; latMax = 38.402;
lonMin = 26.362; lonMax = 26.560;

% Burn threshold
useAutoThreshold = true;      % Otsu on dNBR
dNBR_thr_manual = 0.27;

% Water masking
removeWaterFromBurnMask = true;
ndwiWaterThr = 0.0;          % NDWI > 0 often water (0.0~0.2 tune)

outMat = "A_satellite_outputs.mat";

% LOAD + TRUE GEO-CROP
disp("Reading & cropping BEFORE...");
BEF = readS2_AOI(beforeFolder, before_id, latMin, latMax, lonMin, lonMax);

disp("Reading & cropping AFTER...");
AFT = readS2_AOI(afterFolder, after_id, latMin, latMax, lonMin, lonMax);

disp("Reading & cropping DURING-1...");
DUR1 = readS2_AOI(during1Folder, during1_id, latMin, latMax, lonMin, lonMax);

disp("Reading & cropping DURING-2...");
DUR2 = readS2_AOI(during2Folder, during2_id, latMin, latMax, lonMin, lonMax);

% INDICES
% NDVI = (NIR - Red)/(NIR + Red)
% NBR = (NIR - SWIR)/(NIR + SWIR)
% dNBR = NBR_before - NBR_after (positive = burn severity)
dNDVI = BEF.NDVI - AFT.NDVI;
dNBR = BEF.NBR - AFT.NBR;

% THRESHOLD (AUTO/MANUAL)
if useAutoThreshold
    d = dNBR(:); d = d(isfinite(d));
    d = max(min(d, 1), -1);
    d01 = (d - min(d)) / (max(d)-min(d) + eps);
    lvl = graythresh(d01);
    dNBR_thr = min(d) + lvl*(max(d)-min(d));
else
    dNBR_thr = dNBR_thr_manual;
end

% BURN MASK
burnMask = cleanMask(dNBR > dNBR_thr);

% WATER MASK (NDWI)

% NDWI = (G - NIR)/(G + NIR)
NDWI = (AFT.G - AFT.NIR) ./ (AFT.G + AFT.NIR + eps);
waterMask = cleanMask(NDWI > ndwiWaterThr);

if removeWaterFromBurnMask

```

```

        burnMask = burnMask & ~waterMask;
end
landMask = ~waterMask;

burnPct = 100 * nnz(burnMask) / numel(burnMask);
sevMean = mean(dNBR(burnMask), 'omitnan');

% OVERLAY (VIS)
overlay = AFT.RGB;
overlay(:,:,1) = min(overlay(:,:,1) + 0.70*burnMask, 1);
overlay(:,:,3) = min(overlay(:,:,3) + 0.25*waterMask, 1);

figure('Position',[60 60 1400 750]);
subplot(2,3,1); imshow(BEF.RGB); title("AOI Before RGB");
subplot(2,3,2); imshow(AFT.RGB); title("AOI After RGB");
subplot(2,3,3); imshow(overlay); title("Burn Overlay (red) + Water (blue)");

subplot(2,3,4); imagesc(dNBR); axis image off; colorbar; title("dNBR (AOI)");
subplot(2,3,5); imshow(burnMask); title(sprintf("Burn Mask (%.2f%%)", burnPct));
subplot(2,3,6); imagesc(NDWI); axis image off; colorbar; title("NDWI (AOI)");

% SAVE STRUCT S
S = struct();
S.AOI = struct("latMin",latMin,"latMax",latMax,"lonMin",lonMin,"lonMax",lonMax);

S.BEF = BEF;
S.AFT = AFT;
S.DUR1 = DUR1;
S.DUR2 = DUR2;

S.dNDVI = dNDVI;
S.dNBR = dNBR;
S.dNBR_thr = dNBR_thr;

S.NDWI = NDWI;
S.waterMask = waterMask;
S.landMask = landMask;

S.burnMask = burnMask;
S.burnPct = burnPct;
S.sevMean_dNBR = sevMean;

% Reference for pixel sizing
S.Rref = AFT.Rref;
S.info = AFT.info;

% TABLES (INPUTS + OUTPUTS)
[m,n,~] = size(AFT.RGB);

A_table_inputs = table( ...
    string(before_id), string(after_id), ...
    latMin, latMax, lonMin, lonMax, ...
    useAutoThreshold, dNBR_thr_manual, ...
    removeWaterFromBurnMask, ndwiWaterThr, ...

```

```

    'VariableNames', ["before_id","after_id","latMin","latMax","lonMin","lonMax", ...
        "useAutoThreshold","dNBR_thr_manual","removeWaterFromBurnMask","ndwiWaterThr"] );

A_table_outputs = table( ...
    m, n, nnz(burnMask), burnPct, dNBR_thr, sevMean, ...
    nnz(waterMask), nnz(landMask), ...
    'VariableNames', ["rows_m","cols_n","burnPixels","burnPct","dNBR_thr_used","mean_dNBR_in_burn", ...
        "waterPixels","landPixels"] );

disp("=== Script A Inputs Table ==="); disp(A_table_inputs);
disp("=== Script A Outputs Table ==="); disp(A_table_outputs);

% CALCULATION NOTES
A_calc_notes = struct();
A_calc_notes.steps = {
    "1) Read Sentinel-2 bands (B02,B03,B04,B08 at 10m; B12 at 20m) and geo-crop to AOI."
    "2) Upsample SWIR (B12) to 10m grid to match other bands."
    "3) Compute indices:"
    "    NDVI = (NIR - RED) / (NIR + RED)"
    "    NBR = (NIR - SWIR) / (NIR + SWIR)"
    "    dNBR = NBR_before - NBR_after"
    "4) Threshold dNBR to get burnMask (AUTO Otsu or MANUAL)."
    "5) Compute waterMask using NDWI:"
    "    NDWI = (GREEN - NIR) / (GREEN + NIR)"
    "6) landMask = ~waterMask; burnMask = burnMask & landMask"
};
A_calc_notes.key_formulas = struct( ...
    "NDVI","(NIR - RED)/(NIR + RED)", ...
    "NBR","(NIR - SWIR)/(NIR + SWIR)", ...
    "dNBR","NBR_before - NBR_after", ...
    "NDWI","(GREEN - NIR)/(GREEN + NIR)" );

save(outMat, "S","A_table_inputs","A_table_outputs","A_calc_notes");
disp("Saved: " + outMat);

%% =====
% LOCAL FUNCTIONS
%% =====
function out = readS2_AOI(folder, id, latMin, latMax, lonMin, lonMax)
    fB2 = folder + "\" + id + "_B02_10m.jp2"; % Blue
    fB3 = folder + "\" + id + "_B03_10m.jp2"; % Green
    fB4 = folder + "\" + id + "_B04_10m.jp2"; % Red
    fB8 = folder + "\" + id + "_B08_10m.jp2"; % NIR
    fB12 = folder + "\" + id + "_B12_20m.jp2"; % SWIR (20m)

    [B4, R4] = readgeoraster(fB4);
    [B2, ~] = readgeoraster(fB2);
    [B3, ~] = readgeoraster(fB3);
    [B8, ~] = readgeoraster(fB8);
    [B12, R12] = readgeoraster(fB12);

    info4 = georasterinfo(fB4);

    [r1,r2,c1,c2] = aoWindowFromLatLon(R4, info4, latMin, latMax, lonMin, lonMax, size(B4,1), size(B4,2));

```

```

B2 = B2(r1:r2, c1:c2);
B3 = B3(r1:r2, c1:c2);
B4 = B4(r1:r2, c1:c2);
B8 = B8(r1:r2, c1:c2);

info12 = georasterinfo(fB12);
[r1b,r2b,c1b,c2b] = aoiWindowFromLatLon(R12, info12, latMin, latMax, lonMin, lonMax, size(B12,1), size(B12,2));
B12c = B12(r1b:r2b, c1b:c2b);
B12c = imresize(B12c, size(B4), "bilinear");

scale = 10000;
B2 = single(B2)/scale; B3 = single(B3)/scale; B4 = single(B4)/scale;
B8 = single(B8)/scale; B12c = single(B12c)/scale;

RGB = cat(3, mat2gray(B4), mat2gray(B3), mat2gray(B2));
RGB = imadjust(RGB, stretchlim(RGB), []);

NDVI = (B8 - B4) ./ (B8 + B4 + eps);
NBR = (B8 - B12c) ./ (B8 + B12c + eps);

out = struct();
out.RGB = RGB;
out.NDVI = NDVI;
out.NBR = NBR;

out.R = B4; out.G = B3; out.B = B2;
out.NIR = B8;

out.Rref = R4;
out.info = info4;
end

function [r1,r2,c1,c2] = aoiWindowFromLatLon(R, info, latMin, latMax, lonMin, lonMax, rows, cols)
    latLo = min(latMin, latMax); latHi = max(latMin, latMax);
    lonLo = min(lonMin, lonMax); lonHi = max(lonMin, lonMax);

    if isprop(R, "LatitudeLimits") || contains(class(R), "Geographic")
        [cTL, rTL] = geographicToIntrinsic(R, latHi, lonLo);
        [cBR, rBR] = geographicToIntrinsic(R, latLo, lonHi);
    else
        crs = info.CoordinateReferenceSystem;
        if isempty(crs), error("No CRS found in JP2."); end
        [xTL, yTL] = projfwd(crs, latHi, lonLo);
        [xBR, yBR] = projfwd(crs, latLo, lonHi);
        [cTL, rTL] = worldToIntrinsic(R, xTL, yTL);
        [cBR, rBR] = worldToIntrinsic(R, xBR, yBR);
    end

    r1 = max(1, floor(min(rTL, rBR)));
    r2 = min(rows, ceil(max(rTL, rBR)));
    c1 = max(1, floor(min(cTL, cBR)));
    c2 = min(cols, ceil(max(cTL, cBR)));

    if r2 <= r1 || c2 <= c1

```

```

        error("AOI window invalid. Check bounding box coordinates.");
    end
end

function M = cleanMask(M)
    M = logical(M);
    M = bwareaopen(M, 200);
    M = imclose(M, strel('disk', 5));
    M = imfill(M, 'holes');
end

```

10.2 Meteorological Data Extraction and Integration

```

function B_ERANetCDFwind_simplespreadprediction()
clc; clear; close all;

%%USER SETTINGS
aMat = "A_satellite_outputs.mat"; % Script A output
era5Nc = "C:\Users\yemee\Downloads\project\cdsdata\era5\seferihisar_2025-07-02.nc";

% Start time
t0 = datetime(2025,7,2,9,4,11);
t0.TimeZone = "";

T_hours = 3; % horizon 0..3 hours
dt_hours = 1; % ERA5 often hourly

outMat = "B_wind_spread_output.mat";
%% -----

if ~isfile(aMat), error("Missing %s. Run Script A first.", aMat); end
if ~isfile(era5Nc), error("Missing ERA5 NetCDF: %s", era5Nc); end

A = load(aMat);
if ~isfield(A,"S"), error("Script A MAT must contain struct S."); end
S = A.S;

if ~isfield(S,"AFT") || ~isfield(S.AFT,"RGB")
    error("Script A must save S.AFT.RGB");
end

[m,n,~] = size(S.AFT.RGB);
AOI = S.AOI;

% Detect variable names
info = ncinfo(era5Nc);
varNames = string({info.Variables.Name});

timeVar = pickFirstOrEmpty(varNames, ["valid_time","time","Time","TIMESTAMP"]);
uVar = pickFirstOrEmpty(varNames, ["u10","10u","u","U10","u_component_of_wind"]);

```

```

vVar    = pickFirstOrEmpty(varNames, ["v10","10v","v","V10","v_component_of_wind"]);
t2mVar  = pickFirstOrEmpty(varNames, ["t2m","2t","T2M","temperature_2m"]);
rVar    = pickFirstOrEmpty(varNames, ["r","RH","relative_humidity","r2m","humidity"]);
latVar  = pickFirstOrEmpty(varNames, ["latitude","lat","Latitude","LAT"]);
lonVar  = pickFirstOrEmpty(varNames, ["longitude","lon","Longitude","LON"]);

fprintf("Detected variables:\n");
fprintf("  timeVar = %s\n", showVar(timeVar));
fprintf("  uVar    = %s\n", showVar(uVar));
fprintf("  vVar    = %s\n", showVar(vVar));
fprintf("  rVar    = %s\n", showVar(rVar));
fprintf("  t2mVar  = %s\n", showVar(t2mVar));
fprintf("  latVar  = %s\n", showVar(latVar));
fprintf("  lonVar  = %s\n\n", showVar(lonVar));

if timeVar == "" || uVar == "" || vVar == ""
    error("ERA5 file must contain time + u + v (or equivalents).");
end

hasT = (t2mVar ~= "");
hasR = (rVar    ~= "");

%% Read coords/time
lat = []; lon = [];
if latVar ~= "" && lonVar ~= ""
    lat = double(ncread(era5Nc, char(latVar)));
    lon = double(ncread(era5Nc, char(lonVar)));
end

tRaw = ncread(era5Nc, char(timeVar));
tDT  = convertCFTIMEtoDatetime(era5Nc, char(timeVar), tRaw);
tDT.TimeZone = "";

%% Choose times within [t0, t0+T_hours]
t1 = t0 + hours(T_hours);
idxs = find(tDT >= t0 & tDT <= t1);

if isempty(idx)
    [~,i0] = min(abs(tDT - t0));
    idxs = i0 : min(i0 + round(T_hours/dt_hours), numel(tDT));
end

fprintf("Using %d ERA5 steps from %s to %s\n\n", numel(idxs), string(tDT(idxs(1))), string(tDT(idxs(end))))
Nt = numel(idxs);

%% Preallocate outputs on AOI grid
uA = zeros(m,n,Nt,'single');
vA = zeros(m,n,Nt,'single');
wsA = zeros(m,n,Nt,'single');
wd_toDeg = zeros(m,n,Nt,'single');

if hasT, t2mA = zeros(m,n,Nt,'single'); else, t2mA = []; end
if hasR, rA    = zeros(m,n,Nt,'single'); else, rA    = []; end

```

```

[latGrid, lonGrid] = makeAOIlatlonGrid(AOI, m, n);

%% Read arrays
uAll = ncread(era5Nc, char(uVar));
vAll = ncread(era5Nc, char(vVar));
if hasT, tAll = ncread(era5Nc, char(t2mVar)); end
if hasR, rAll = ncread(era5Nc, char(rVar)); end

for k = 1:Nt
    it = idxs(k);

    U = squeeze(sliceAtTime(uAll, it));
    V = squeeze(sliceAtTime(vAll, it));

    if hasT
        T = squeeze(sliceAtTime(tAll, it));
    else
        T = [];
    end

    if hasR
        RH = squeeze(sliceAtTime(rAll, it));
    else
        RH = [];
    end

    [u2, v2, t2, r2] = alignERA5toAOI(U, V, T, RH, lat, lon, latGrid, lonGrid, m, n);

    uA(:, :, k) = single(u2);
    vA(:, :, k) = single(v2);

    ws = hypot(double(u2), double(v2));
    wsA(:, :, k) = single(ws);

    wd_to = mod(atan2d(double(u2), double(v2)), 360);
    wd_toDeg(:, :, k) = single(wd_to);

    if hasT, t2mA(:, :, k) = single(t2); end
    if hasR, rA(:, :, k) = single(r2); end
end

%% Compatibility fields for Script C
u = uA(:, :, 1);
v = vA(:, :, 1);
ws = wsA(:, :, 1);
wd_toDeg0 = wd_toDeg(:, :, 1);

%% Tables (inputs + outputs)
B_table_inputs = table( ...
    string(era5Nc), string(aMat), t0, T_hours, dt_hours, ...
    "VariableNames", ["era5Nc", "aMat", "t0", "T_hours", "dt_hours"]);

B_table_outputs = table( ...

```

```

    Nt, tDT(idxs(1)), tDT(idxs(end)), ...
    min(wsA,[],"all"), mean(wsA,"all"), max(wsA,[],"all"), ...
    hasT, hasR, ...
    "VariableNames", ["Nt","tStart","tEnd","ws_min","ws_mean","ws_max","hasT","hasR"]);

disp("=== Script B Inputs ==="); disp(B_table_inputs);
disp("=== Script B Outputs ==="); disp(B_table_outputs);

save(outMat, ...
    "t0","T_hours","dt_hours","idxs","tDT", ...
    "uA","vA","wsA","wd_toDeg", ...
    "u","v","ws","wd_toDeg0", ...
    "t2mA","rA","hasT","hasR", ...
    "lat","lon","AOI","m","n", ...
    "B_table_inputs","B_table_outputs");

disp("Saved: " + outMat);
end

%% FUNCTIONS
function s = showVar(v)
if v == "", s="(not found)"; else, s=char(v); end
end

function name = pickFirstOrEmpty(varNames, candidates)
candidates = string(candidates);
name = "";
for k = 1:numel(candidates)
    if any(varNames == candidates(k))
        name = candidates(k);
        return;
    end
end
end

function tDT = convertCFTTimeToDatetime(ncFile, timeVarName, tRaw)
units = "";
try, units = string(ncreadatt(ncFile, timeVarName, "units")); catch, end
tRaw = double(tRaw(:));

if strlength(units)==0
    tDT = datetime(1970,1,1,0,0,0) + seconds(tRaw);
    tDT.TimeZone = "";
    return;
end

u = lower(strtrim(units));
parts = split(u, "since");
step = strtrim(parts(1));
originStr = "1970-01-01 00:00:00";
if numel(parts) >= 2, originStr = strtrim(parts(2)); end
origin = parseOriginDatetime(originStr);

switch step

```

```

        case {"hours","hour"},    tDT = origin + hours(tRaw);
        case {"minutes","minute"}, tDT = origin + minutes(tRaw);
        case {"seconds","second","sec","secs"}, tDT = origin + seconds(tRaw);
        case {"days","day"},      tDT = origin + days(tRaw);
        otherwise,                tDT = origin + hours(tRaw);
    end
    tDT.TimeZone = "";
end

function origin = parseOriginDatetime(s)
s = char(strtrim(string(s)));
fmts = {'yyyy-MM-dd HH:mm:ss','yyyy-MM-dd''T''HH:mm:ss','yyyy-MM-dd HH:mm','yyyy-MM-dd','yyyy-MM-dd''T''
for i=1:numel(fmts)
    try, origin = datetime(s, "InputFormat", fmts{i}, "TimeZone",""); origin.TimeZone=""; return; catch
end
origin = datetime(s); origin.TimeZone="";
end

function slice2D = sliceAtTime(A, idxTime)
sz = size(A);
nd = ndims(A);
if nd == 2
    slice2D = A; return;
end
for d = 1:nd
    if idxTime <= sz(d)
        subs = repmat({' ':''}, 1, nd);
        subs{d} = idxTime;
        try
            tmp = squeeze(A(subs{:}));
            if ismatrix(tmp), slice2D = tmp; return; end
        catch
        end
    end
end
error("Could not slice array at time index %d", idxTime);
end

function [latGrid, lonGrid] = makeAOIlatlonGrid(AOI, m, n)
latVec = linspace(AOI.latMax, AOI.latMin, m);
lonVec = linspace(AOI.lonMin, AOI.lonMax, n);
[lonGrid, latGrid] = meshgrid(lonVec, latVec);
end

function [u2,v2,t2,r2] = alignERA5toAOI(U,V,T,RH, lat, lon, latGrid, lonGrid, m, n)
t2 = []; r2 = [];

if isempty(lat) || isempty(lon) || numel(lat)<2 || numel(lon)<2 || min(size(U))<2
    u2 = repmat(double(U(1)), m, n);
    v2 = repmat(double(V(1)), m, n);
    if ~isempty(T), t2 = repmat(double(T(1)), m, n); end
    if ~isempty(RH), r2 = repmat(double(RH(1)), m, n); end
    return;
end
end

```

```

latv = lat(:); lonv = lon(:);
U = double(U); V = double(V);

if latv(1) > latv(end)
    latv = flipud(latv);
    U = flipud(U); V = flipud(V);
    if ~isempty(T), T = flipud(double(T)); else, T=[]; end
    if ~isempty(RH), RH = flipud(double(RH)); else, RH=[]; end
else
    if ~isempty(T), T = double(T); end
    if ~isempty(RH), RH = double(RH); end
end

FU = griddedInterpolant({latv, lonv}, U, "linear", "none");
FV = griddedInterpolant({latv, lonv}, V, "linear", "none");

u2 = fillmissing(FU(latGrid, lonGrid), "nearest");
v2 = fillmissing(FV(latGrid, lonGrid), "nearest");

if ~isempty(T)
    FT = griddedInterpolant({latv, lonv}, T, "linear", "none");
    t2 = fillmissing(FT(latGrid, lonGrid), "nearest");
end
if ~isempty(RH)
    FR = griddedInterpolant({latv, lonv}, RH, "linear", "none");
    r2 = fillmissing(FR(latGrid, lonGrid), "nearest");
end
end
end

```

10.3 Fire Spread Prediction Model

```

function C_Realistic_Final()
clc; clear; close all;

%% 1. LOAD DATA & SYNC TIMING
aMat = "A_satellite_outputs.mat";
bMat = "B_wind_spread_output.mat";

if ~isfile(aMat) || ~isfile(bMat)
    error("Missing input files. Please ensure Script A and B have been run and saved.");
end

A = load(aMat); B = load(bMat);
S = A.S;
[m, n, ~] = size(S.AFT.RGB);

% Set Simulation Start Time
t0_sim = datetime(2025,7,2,9,4,11);

%% 2. SIMULATION SETTINGS
T_hours = 4.0;
dt_minutes = 15;

```

```

nSteps      = round(T_hours*60/dt_minutes) + 1;
tVec_h      = (0:nSteps-1)' * (dt_minutes/60);
dt_s        = dt_minutes * 60;

baseRate_mps = 0.008;
windGain_mps = 0.005;
ellipticity  = 2.5;
anisoStrength = 0.85;
dx = 10;

%% 3. INITIALIZE MASKS & GROWTH TRACKER
predMasks = false(m, n, nSteps);

if isempty(S.burnMask) || nnz(S.burnMask) == 0
    error("The burn mask from Script A is empty. Check your dNBR threshold.");
end

predMasks(:,:,1) = logical(S.burnMask);
landMask = ~(S.waterMask);

% Initialize burned area array (m2)
burnedArea_m2 = zeros(nSteps, 1);
pixelArea_m2 = dx * dx;
burnedArea_m2(1) = nnz(predMasks(:,:,1)) * pixelArea_m2;

% Prepare ERA5 timeline alignment
era5_t_hours = hours(datetime(2025,7,2,9,0,0) + hours(0:size(B.uA,3)-1) - t0_sim);

fprintf("Starting Real-Time Wind Integration...\n");

%% 4. SIMULATION LOOP
for k = 2:nSteps
    t_curr = tVec_h(k);

    % Interpolate wind data for current time step
    u_raw = interpTimeStack(B.uA, era5_t_hours, t_curr);
    v_raw = interpTimeStack(B.vA, era5_t_hours, t_curr);

    % Resize wind map to match satellite 10m grid
    u_map = imresize(u_raw, [m, n], 'bilinear');
    v_map = imresize(v_raw, [m, n], 'bilinear');

    % Calculate wind speed and unit vectors
    ws = hypot(u_map, v_map);
    ux = u_map ./ (ws + eps);
    uy = -v_map ./ (ws + eps);

    % Calculate step distance based on local wind speed
    stepDist_m = (baseRate_mps + windGain_mps * ws) * dt_s;

    % Propagate fire front
    prev = predMasks(:,:,k-1);

```

```

    next = propagateEfficient(prev, stepDist_m, ux, uy, ellipticity, anisoStrength, dx);

    % Constrain to land (remove water)
    predMasks(:,:,k) = next & landMask;

    % Track growth
    burnedArea_m2(k) = nnz(predMasks(:,:,k)) * pixelArea_m2;

    if mod(k,4)==0
        fprintf(' T+%.1f hours | Mean Wind Speed: %.2f m/s\n', t_curr, mean(ws,'all'));
    end
end

%% 5. DIAGNOSTICS & OUTPUT
actualPixels = nnz(logical(S.burnMask));
predPixels = nnz(predMasks(:,:,end));
intersection = predMasks(:,:,end) & logical(S.burnMask);

% Accuracy Metrics
iou = (nnz(intersection) / (actualPixels + predPixels - nnz(intersection))) * 100;
growth_error = ((predPixels - actualPixels) / actualPixels) * 100;
totalGrowth_m2 = burnedArea_m2(end) - burnedArea_m2(1);

% FIXED PRINTING BLOCK
fprintf('\n===== \n');
fprintf('      FIRE SPREAD ANALYSIS RESULTS      \n');
fprintf('===== \n');
fprintf('Initial Area:      %15.2f m^2\n', burnedArea_m2(1));
fprintf('Final Area:        %15.2f m^2\n', burnedArea_m2(end));
fprintf('----- \n');
fprintf('TOTAL GROWTH:      %15.2f m^2\n', totalGrowth_m2);
fprintf('----- \n');
fprintf('Final IoU Score: %15.2f %%\n', iou);
fprintf('Area Error:        %15.2f %%\n', growth_error);
fprintf('===== \n');

%% 6. VISUALIZATION
figure('Name','Spatial Validation','Position',[100 100 1200 500], 'Color', 'w');
subplot(1,2,1);
imshow(S.AFT.RGB); hold on;
visboundaries(predMasks(:,:,end), 'Color', 'r', 'LineWidth', 2);
title(sprintf('Model Prediction (Red) | IoU: %.1f%%', iou));

subplot(1,2,2);
imshow(S.AFT.RGB); hold on;
visboundaries(logical(S.burnMask), 'Color', 'g', 'LineWidth', 2);
title('Satellite Observation (Green Truth)');

figure('Name','Area Growth Over Time','Position',[200 200 800 400], 'Color', 'w');
plot(tVec_h, burnedArea_m2, '-o', 'Color', [0.85 0.33 0.1], 'LineWidth', 2, 'MarkerSize', 4);
grid on;
xlabel('Simulation Time (Hours)');
ylabel('Burned Area (m^2)');
title('Simulated Fire Growth Rate');

```

```
% Save final workspace results
save("C_realistic_results.mat", "predMasks", "iou", "burnedArea_m2", "tVec_h");
fprintf('Results saved to C_realistic_results.mat\n');
```

```
end
```

```
%% FUNCTIONS
```

```
function next = propagateEfficient(prev, dist_m, ux, uy, ellip, aniso, dx)
```

```
    [m, n] = size(prev);
    a_px = (dist_m / dx) .* (1 + aniso*(ellip-1));
    b_px = (dist_m / dx);
    base = bwdist(prev) <= mean(b_px, 'all');
    [rr, cc] = ndgrid(1:m, 1:n);
    push_px = max(0, a_px - b_px);
    rr2 = round(rr - uy .* push_px);
    cc2 = round(cc - ux .* push_px);
    valid = (rr2 >= 1 & rr2 <= m & cc2 >= 1 & cc2 <= n);
    adv = false(m, n);
    idx = sub2ind([m, n], rr2(valid), cc2(valid));
    adv(valid) = prev(idx);
    next = base | adv;
```

```
end
```

```
function val = interpTimeStack(stack, t_vec, t_target)
```

```
    idx = find(t_vec <= t_target, 1, 'last');
    if isempty(idx), idx = 1; end
    if idx >= numel(t_vec)
        val = stack(:, :, end);
    else
        f = (t_target - t_vec(idx)) / (t_vec(idx+1) - t_vec(idx));
        val = (1-f)*stack(:, :, idx) + f*stack(:, :, idx+1);
    end
```

```
end
```

```
end
```

10.4 Reporting

```
%clc, clear, close all
```

```
%%% COST-231 path loss & received power
```

```
%
```

```
%f = 800e6;          % Frequency (Hz)
```

```
%d = linspace(1e3, 30e3, 100); % 1{30 km
```

```
%Pt = 30;           % Tx Power (dBm)
```

```
%Gt = 5;            % Tx Gain (dBi)
```

```
%Gr = 15;           % Rx Gain (dBi)
```

```
%
```

```
%ht = 30;          % Tx height (m)
```

```
%hr = 10;          % Rx height (m)
```

```
%
```

```
%a_hr = (1.1*log10(f/1e6) - 0.7)*hr - (1.56*log10(f/1e6) - 0.8);
```

```
%
```

```
%PL = 46.3 + 33.9*log10(f/1e6) - 13.82*log10(ht) - a_hr + (44.9 - 6.55*log10(ht))*log10(d/1e3);
```

```
%Pr = Pt + Gt + Gr - PL;
```

```
%
```

```

%% LTE sensitivity and margin
%Psens = -100;          % dBm
%margin = 10;          % dB
%Psafe = Psens + margin;
%
%% Find distances
%d0 = interp1(Pr, d, Psens);    % 0 dB margin distance
%d10 = interp1(Pr, d, Psafe);   % +10 dB margin distance
%
%figure
%plot(d/1e3, Pr, 'LineWidth',2)
%hold on
%yline(Psens, 'r--', 'LTE Sensitivity (-100 dBm)', 'LineWidth',1.5)
%yline(Psafe, 'g--', '+10 dB Margin (-90 dBm)', 'LineWidth',1.5)
%
%plot(d0/1e3, Psens, 'ro', 'MarkerSize',8, 'MarkerFaceColor', 'r')
%plot(d10/1e3, Psafe, 'go', 'MarkerSize',8, 'MarkerFaceColor', 'g')
%
%text(d0/1e3, Psens+2, sprintf('%.1f km (0 dB)', d0/1e3))
%text(d10/1e3, Psafe+2, sprintf('%.1f km (+10 dB)', d10/1e3))
%
%xlabel("Distance (km)")
%ylabel("Received Power (dBm)")
%title("LTE NLOS Link Budget with Link Margin")
%grid on
%
%disp(['Received Power = ', num2str(Pr), ' dBm'])
%
%% BER vs SNR (QPSK)
%snr = 0:1:20;
%ber = berawgn(snr, 'psk',4, 'nondiff');
%figure
%semilogy(snr,ber, 'LineWidth',2)
%grid on
%xlabel('SNR (dB)')
%ylabel('BER')
%title('QPSK Performance for Fire Alert Link')
%
%% Antenna Radiation Pattern (Tx & Rx)
%theta = linspace(0,2*pi,1000);
%
%TxPattern = Gt * abs(cos(theta));
%RxPattern = Gr * abs(cos(theta).^4);
%
%figure
%polarplot(theta, TxPattern, 'LineWidth',2)
%hold on
%polarplot(theta, RxPattern, 'LineWidth',2)
%title('Antenna Radiation Patterns at 800 MHz')
%legend('Tx Omnidirectional', 'Rx Directional')
%
%% Constellation Diagram
%data = randi([0 3],1000,1);
%modSig = pskmod(data,4,pi/4);

```

```

%
%figure
%scatterplot(modSig)
%title('QPSK Constellation Diagram for Alert Transmission')
%
%%% Alert triggering based on confidence and burned area
%C_min = 0.75;          % minimum detection confidence
%A_min = 1e4;           % minimum burned area (m^2)
%
%C = 0.82;              % example detection confidence
%A = 2.3e4;             % example burned area
%
>alertFlag = (C >= C_min) && (A >= A_min);
%
%fprintf("Alert Triggered: %d\n", alertFlag);
%
%%% 3D Antenna Radiation Patterns
%
%
%%% Angular grids
%theta = linspace(0, pi, 180);      % elevation
%phi    = linspace(0, 2*pi, 360);   % azimuth
%[TH, PH] = meshgrid(theta, phi);
%
%%% Antenna gains (linear scale)
%Gtlin = 10^(5/10);      % Tx gain ~ 5 dBi
%Grin  = 10^(15/10);     % Rx gain ~ 15 dBi
%
%%% Radiation patterns
%TxPattern = Gtlin * abs(sin(TH));   % quasi-omnidirectional
%TH_tilt = TH - deg2rad(5);
%RxPattern = Grin * abs(cos(TH_tilt)).^4; % narrow beam directional
%
%%% Convert to Cartesian coordinates
%[Xt,Yt,Zt] = sph2cart(PH, pi/2 - TH, TxPattern);
%[Xr,Yr,Zr] = sph2cart(PH, pi/2 - TH, RxPattern);
%
%
%%% Plot Transmit Antenna Pattern
%
%figure
%surf(Xt, Yt, Zt, TxPattern, 'EdgeColor','none')
%axis equal
%title('3D Transmit Antenna Radiation Pattern (Alert Node)')
%xlabel('X'); ylabel('Y'); zlabel('Z')
%colorbar
%view(45,30)
%lighting gouraud
%camlight headlight
%
%%% =====
%%% Plot Receive Antenna Pattern
%%% =====
%figure

```

```
%surf(Xr, Yr, Zr, RxPattern, 'EdgeColor','none')
%axis equal
%title('3D Receive Antenna Radiation Pattern (AFAD Base Station)')
%xlabel('X'); ylabel('Y'); zlabel('Z')
%colorbar
%view(45,30)
%lighting gouraud
%camlight headlight
```